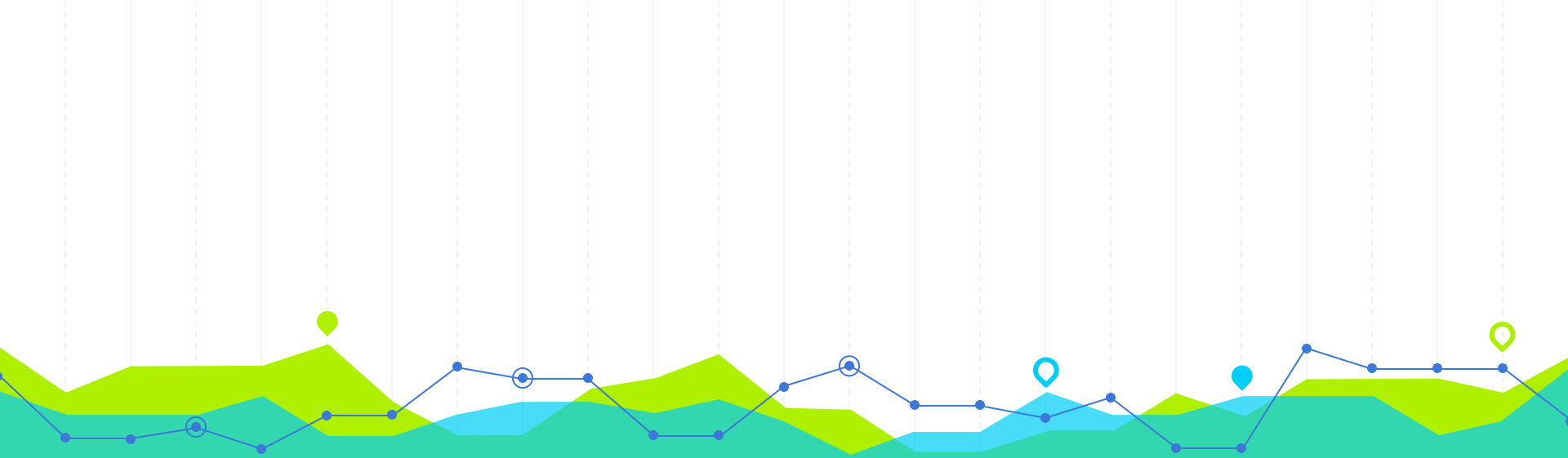
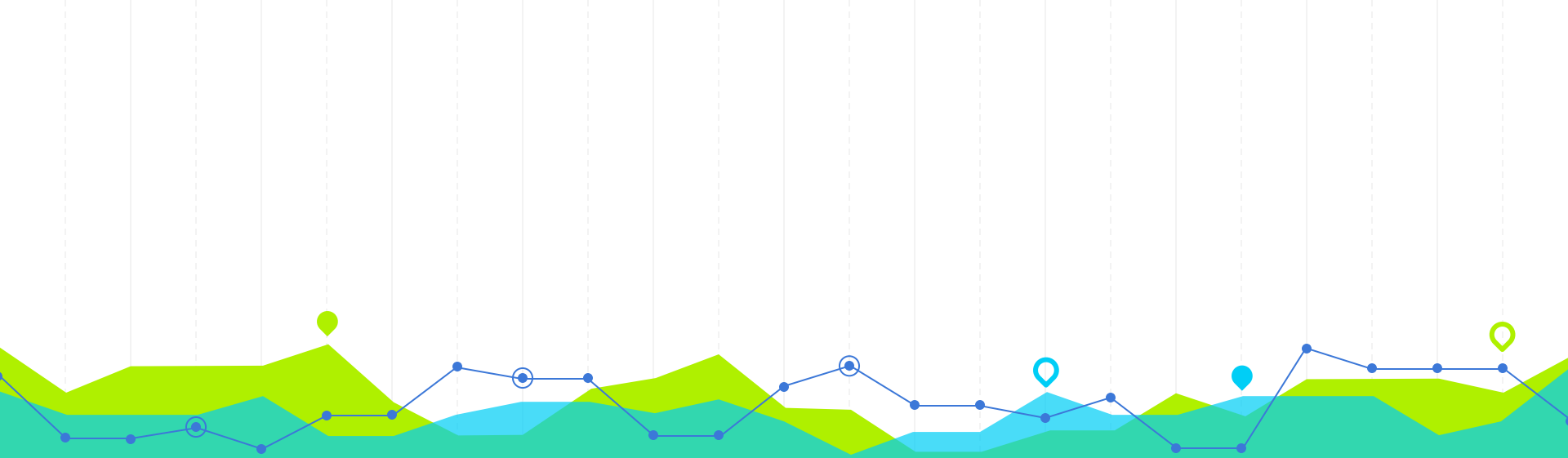




Coder en Python

pour la production



**Coder avec les
bonnes pratiques**

	J2
8:30 - 10:00	Coder avec les bonnes pratiques
10:00 - 10:15	
10:15 - 11h:45	

Le programme

Coder avec les bonnes pratiques

Objectifs

- écrire du code propre, performant, robuste et de qualité

Programme

1. Introduction
2. Code **clean** avec les guidelines **pep8**
3. Gérer les **Exceptions**
4. **Logger** son application
5. **Sécuriser** son application
6. **Documenter** son code
7. **Tester** son code



Objectif

Coder avec les bonnes pratiques

Un code de qualité
= un travail de qualité





1. Introduction

Qu'est ce qu'un code de qualité ?

Les caractéristiques du code de qualité :

- **lisibilité** : **facile à lire** pour vous et d'autres développeurs
- **maintenabilité** : **structuré** -> facile à modifier, à étendre ou à déboguer
- **performance** : **optimisé** pour offrir de bonnes performances
- **fiabilité** : **robuste**, avec un **minimum d'erreurs** et de **bugs**
- **réutilisabilité** : pouvoir être **réutilisé** dans d'autres projets / contextes



Pourquoi faire un code de qualité ?



Les coûts cachés d'un code de **faible qualité**

- **temps de débogage** : bugs plus difficiles à localiser et à corriger
- **coûts de maintenance élevés** avec des modifications / ajouts de fonctionnalités compliqués et longs
- **difficultés de collaboration** si les nouveaux membres de l'équipe ont du mal à comprendre le code
- **mauvaise réputation** pour l'entreprise et/ou le(s) développeur(s) si les produits sont basés sur un code de faible qualité
- **risque technique** : Augmentation du risque de pannes ou de problèmes de sécurité

Code qui "fonctionne" VS code de production

Code qui "fonctionne"

Code qui accomplit une tâche mais pourrait ne pas être optimisé, bien structuré ou documenté.

Il peut **manquer de tests** et pourrait **ne pas gérer tous les cas d'usage** ou les **erreurs** possibles. **Non maintenable sur la durée.**

Code "prêt pour la production"

Code **robuste**, **efficace**, **testé** et **documenté**. Est conçu pour être **maintenu** et **évolué** à long terme, avec une attention particulière à la **prévention des bugs** et à l'**efficacité opérationnelle**.

Dev:

Ca marche sur mon pc



PM:

Oui mais on va pas donner ton pc au client



Cédric Compagnon.

Différences entre le code pour le développement rapide (prototypage) et le code de production



YOUR MODEL IN
A JUPYTER
NOTEBOOK

YOUR MODEL IN
PRODUCTION

	Prototypage	Code de production
Objectif	Valider rapidement une idée => "faire en sorte que ça fonctionne" plutôt que "faire ça bien" => Quick and dirty	Fiabilité, efficacité, maintenance, sécurité et robustesse à toutes sortes de scénarios d'utilisation réels
Qualité et robustesse	Peut comporter des raccourcis, des "hacks" temporaires, ou ne pas gérer tous les cas d'erreur possibles	Doit être soigneusement testé, optimisé et débogué. Tout scénario d'erreur doit être anticipé et correctement géré.
Maintenance	Écrit rapidement, sans trop se soucier des conventions ou de la documentation. Pas conçu pour être maintenu sur le long terme.	Écrit en gardant à l'esprit la maintenance. Il est bien documenté, suit des conventions de codage strictes et revu par d'autres développeurs (code reviews)
Performance	Pas une priorité	Cruciale, notamment pour des applications à grande échelle
Sécurité	Minimisée ou ignorée	Essentielle. Les vulnérabilités doivent être identifiées et corrigées
Extensibilité	pour des besoins spécifiques	conçu pour être extensible

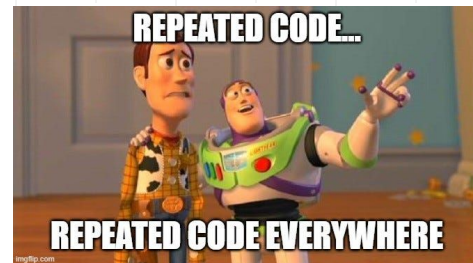
Importance de la modularité et de la réutilisabilité



La **modularité** et la **réutilisabilité** sont deux concepts essentiels en programmation

- Diviser un programme complexe en petits modules / fonctions rend le code **plus lisible** / **facile** à comprendre
- Les fonctions **ne doivent pas avoir d'effets secondaires**
Elles doivent prendre des entrées et produire des sorties sans modifier d'états globaux
- **Réutilisabilité** : un module est réutilisable dans plusieurs parties du programme ou dans d'autres programmes
- **Maintenance plus facile** : si bug est découvert dans un module, il suffit de le corriger juste dans le module
- **Collaboration améliorée** : la modularité permet à chaque développeur de travailler sur un module différent
- La réutilisabilité permet d'**éviter la duplication de code**
- Avec des modules bien définis, il est plus facile d'identifier les goulots d'étranglement et de les **optimiser**
- Un système modulaire est **adaptable** / **évolutif** sans perturber le fonctionnement des modules existants

Principe DRY (Don't Repeat Yourself)



Objectif : favoriser l'**abstraction** (sans aller à l'extrême non plus) pour **factoriser** et éviter de dupliquer le code

En respectant le principe DRY :

- Votre code est plus facile à **maintenir**
- **Réduction** des chances d'introduire des **bugs** lors des modifications
- Le code est plus **lisible** / **concis**
- **Ne nécessite pas de** modifications multiples si une logique doit être changée

Faire un copier/coller met en péril ce principe 😬



Principes SOLID

S - Responsabilité unique (Single Responsibility Principle) : chaque classe ou fonction doit avoir une seule mission. Elle fait une seule chose bien.

O - Ouvert/Fermé (Open/Closed Principle) : encourage l'héritage et la surcharge pour modifier le comportement d'une classe sans la modifier directement (pour ne pas modifier ce qui marche déjà)

L - Substitution de Liskov (Liskov Substitution Principle) : si une classe B hérite d'une classe A, alors nous devrions être capables d'utiliser B partout où nous utilisons A, sans rencontrer de problèmes

I - Ségrégation des interfaces (Interface Segregation Principle) : Mieux vaut avoir **plusieurs petites interfaces** spécifiques plutôt qu'une grande interface avec trop de méthodes inutiles

D - Inversion des dépendances (Dependency Inversion Principle) : le code ne doit pas dépendre d'implémentations spécifiques, mais plutôt d'abstractions (comme des interfaces ou des classes abstraites). Les composants sont ainsi plus flexibles et peuvent être facilement remplacés ou modifiés sans toucher au reste du code.

SOLID aide à rendre le code plus propre, plus facile à modifier et plus stable sur le long terme

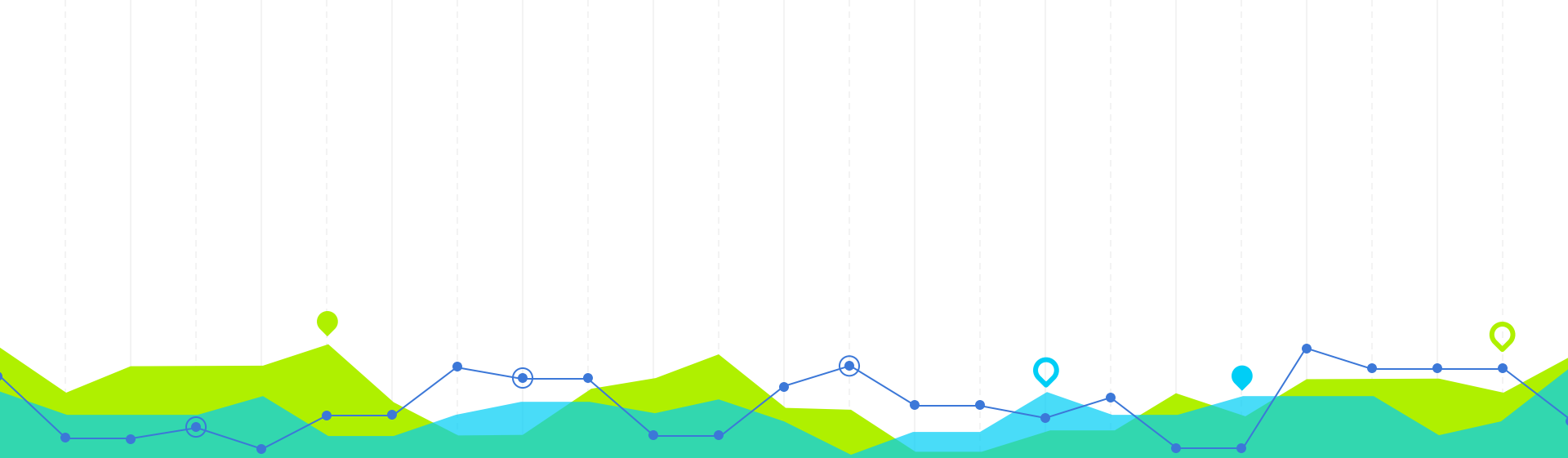
when you finish coding
so you can close your
200 tabs

Conclusion



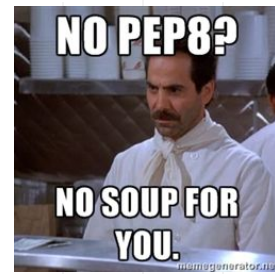
Le but n'est pas de viser la perfection absolue, ce qui serait souvent **contre-productif** en termes de **coûts** et de **temps**.

Il faut trouver un **équilibre** où le code est de **bonne qualité** tout en respectant les **contraintes** de **temps** et de **budget**.



2. Code clean avec les guidelines pep8

Qu'est ce que PEP ?

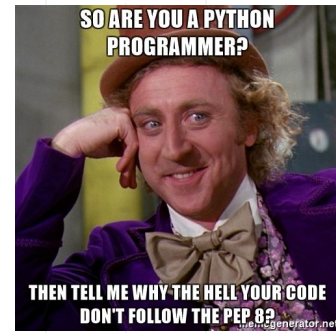


Le terme "PEP" signifie "Python Enhancement Proposal". Il s'agit d'un ensemble de propositions de design pour améliorer le langage Python, ses bibliothèques ou ses processus de développement.

1. **PEP 8** - guide du style de code pour Python : décrit les conventions de style pour le code Python. Il couvre des sujets tels que la manière d'indenter le code, comment nommer les variables et les fonctions, comment organiser les imports, etc.
2. **PEP 20** - Le Zen de Python : liste de 19 aphorismes (le 20ème est laissé intentionnellement vide) qui capturent la philosophie du langage Python. Vous pouvez le voir en tapant `import this` dans un interpréteur Python.
3. **PEP 257** - conventions relatives aux docstrings : donne des directives sur comment écrire des "docstrings"
4. **PEP 484** - Typing hints (Indications de type) : Introduit la notion d'indications de type dans le langage
5. etc.

Il existe de nombreux autres PEPs couvrant une variété de sujets, de la syntaxe du langage à la bibliothèque standard, en passant par les processus de développement. Si vous êtes intéressé par un aspect particulier du langage ou de son écosystème, il y a probablement un PEP qui en traite !

Qu'est ce que PEP 8 ?



PEP 8 est le **guide de style pour le code Python**.

"PEP" signifie "Python Enhancement Proposal" et le numéro "8" correspond à l'identifiant de ce PEP spécifique.

L'idée principale derrière PEP 8 est de rendre le code Python **plus lisible et uniforme**, facilitant ainsi la **maintenance** et la **collaboration**.

PEP 8 **n'est pas obligatoire**, mais c'est une **bonne pratique**, surtout si vous travaillez avec d'autres développeurs.

il peut être justifié de ne pas suivre certaines recommandations de PEP 8, notamment lorsque le respect de ces recommandations rendrait le code moins lisible.

"Face à l'ambiguïté, refusez la tentation de deviner.", PEP8

Qu'inclut pep8 ?



- **Mise en forme du code:** indentation de 4 espaces, taille maximale des lignes
- **Nomination des variables:** *snake_case* plutôt que *camelCase*, `CONST_NAME` pour les constantes
- **Importation de modules:** trois sections séparées d'une ligne vide: librairies standards, librairies externes, librairies internes
- **Commentaires:** doivent être séparés avec deux espaces du code
- **Docstrings:** au début de chaque fonction, module, classe ou méthode, entre *triple quotes*
- **etc.**

Mise en forme du code

Indentation

- Respecter la convention de **4 espaces** en Python
- Ne **pas mélanger** tabs et espaces
- limiter les lignes à **79 caractères** pour le code et 72 pour les commentaires et les docstrings

Bloc de code

- Eviter les fonctions ou méthodes trop longues => **modularité**
- **2 lignes vides** entre chaque classe
- **1 ligne vide** entre chaque méthode de classe
- **1 ligne vide** pour séparer les parties claires de code

Découpage en modules et packages

- Organiser le code en modules selon sa **fonctionnalité**.
- Utiliser des **packages** pour regrouper les modules apparentés.

Encodage : ne jamais utiliser autre chose que de l'UTF-8.

```
class MyClass:
    def first_method(self):
        return None

    def second_method(
        arg_one, arg_two,
        arg_three, arg_four):
        (...) # clear step here

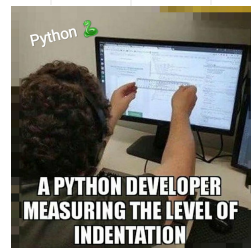
        (...) # other clear step here

    return arg_one
```

```
class MySecondClass:
    pass
```

```
def top_level_function():
    name = 'Batman' # Recommended
    name='name' # Not recommended
    batmobile ['color'] = 'black' # Not recommended
    batmobile['color'] = 'black' # Recommended
    list_of_numbers = [
        1, 2, 3,
        4, 5, 6,
        7, 8, 9
    ] # Recommended
    # Recommended
    var1 = 5
    var2 = 6
    some_long_var = 7
    # Not recommended
    var1     = 5
    var2     = 6
    some_long_var = 7

    return None
```





Nomination des variables

Importance du nommage

- Premier indicateur de la fonction d'une variable ou d'une fonction
- Réduit le besoin de commentaires explicatifs

Conventions

- Variables : snake_case (ex : user_name, total_amount)
- Fonctions et méthodes : snake_case (ex : calculate_total(), parse_input())
- Classes : PascalCase (ex : UserAccount, InvoiceDetail)
- Constante : UPPERCASE (ex: MAX_ITEMS, LABEL)
- Module : snake_case (ex : linear_model, utils)
- Package : lowercase sans séparateur (ex: sqlalchemy, beautifulsoup4)

Descriptivité

- Choisir des noms qui **reflètent la fonction et la signification** (ex : get_average_grade() plutôt que calc_num())
- Éviter les **noms génériques** tels que data, df, info, do_this()
- **Taille** ni trop long ni trop court. Il faut que ce soit explicite tout en restant **succinct**.
- N'utilisez jamais les lettres l, O ou I, elles peuvent être confondues avec 1 et 0
`0 = 2 # This may look like you're trying to reassign 2 to zero`

```
# Not recommended
x = 'John Smith'
y, z = x.split()
print(z, y, sep=', ')
# Affiche 'Smith, John'
```

```
# Recommended
name = 'John Smith'
first_name, last_name = name.split()
print(last_name, first_name, sep=', ')
# Affiche 'Smith, John'
```

Recommandations de programmation



```
def foo(x):  
    if x >= 0:  
        return math.sqrt(x)  
    else:  
        return None # add an explicit return statement  
  
def bar(x):  
    if x < 0:  
        return None  
    return math.sqrt(x)
```

Bad

```
# create a concatenated string from 0 to 19 (e.g. "012..1819")  
nums = ""  
for n in range(20):  
    nums += str(n) # slow and inefficient  
print(nums)
```

Better

```
# create a concatenated string from 0 to 19 (e.g. "012..1819")  
nums = []  
for n in range(20):  
    nums.append(str(n))  
print("".join(nums)) # much more efficient
```

Best

```
# create a concatenated string from 0 to 19 (e.g. "012..1819")  
nums = [str(n) for n in range(20)]  
print("".join(nums))
```

Opérateur := (Morse / Walrus)

Depuis Python 3.8 en octobre 2019, il existe un nouvel opérateur communément appelé opérateur morse (Walrus operator)

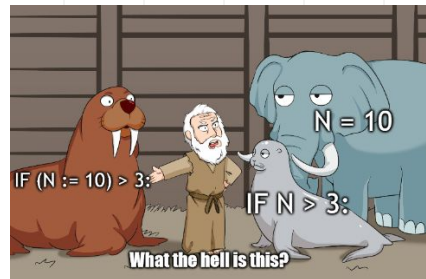
`:=` permet de simultanément faire une **allocation** et un **test** dans la même expression :

Before

```
init_list = ["Python", "PEP", "conda"]  
filtered_list = [x.upper() for x in liste_init if "P" in x.upper()]
```

After

```
init_list = ["Python", "PEP", "conda"]  
filtered_list = [y for x in liste_init if "P" in (y:=x.upper())]
```



```
# Before  
fichier_python = open("python.txt")  
# on fait une boucle sur les lignes jusqu'à  
# trouver le mot python  
line = fichier_python.readline()  
while line:  
    if "python" in line:  
        print("On a trouvé python")  
        break  
    line = fichier_python.readline()
```

```
# After  
fichier_python = open("python.txt")  
# on fait une boucle sur les lignes jusqu'à  
# trouver le mot python  
while line := fichier_python.readline():  
    if "python" in line:  
        print("On a trouvé python")  
        break
```

Importation de modules

Organisation des imports

- Regroupés en en-tête, par ordre de type de bibliothèque et par ordre alphabétique :
 1. bibliothèques standard
 2. bibliothèques tierces
 3. modules locaux
- Utilisez des alias clairs pour les bibliothèques

Very bad

```
[...]
from modu import *
[...]
x = sqrt(4) # Is sqrt part of modu? A builtin? Defined above?
```

Better

```
from modu import sqrt
[...]
x = sqrt(4) # sqrt may be part of modu, if not redefined in between
```

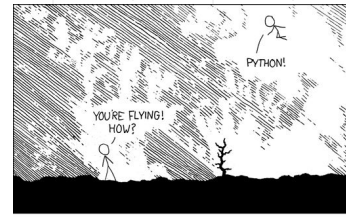
Best

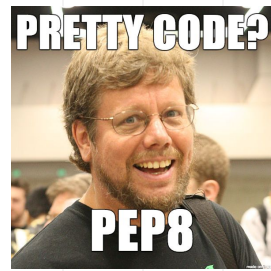
```
import modu
[...]
x = modu.sqrt(4) # sqrt is visibly part of modu's namespace
```

```
import ast
import os
from subprocess import Popen, PIPE
import sys
```

```
import bs4
from pandas as pd
```

```
from foo.bar.yourclass import YourClass
from myclass import MyClass
```





Commentaires

Quand commenter ?

- Lorsque le code est complexe ou non-intuitif
- Lors de l'utilisation de "hacks" ou de solutions temporaires
- Pour documenter des décisions importantes de conception

Comment commenter ?

- Être clair et concis
- Éviter les commentaires redondants (ceux qui expliquent ce que le code fait déjà évidemment)
- Utiliser des **docstrings** pour documenter les fonctions, méthodes et classes
- Recommander d'utiliser l'**anglais** (sauf exceptions)

Éviter les commentaires obsolètes

- Mettre à jour les commentaires si le code change
- Supprimer les commentaires pour le code qui a été supprimé

```
# Not recommended
x = 'John Smith' # Student Name

# Recommended
student_name = 'John Smith'

def quadratic(a, b, c, x):
    # Calculate the solution to a quadratic equation
    # using the quadratic formula.
    #
    # There are always two solutions to a quadratic
    # equation, x_1 and x_2.
    x_1 = (- b + (b**2 - 4*a*c)**(1/2)) / (2*a)
    x_2 = (- b - (b**2 - 4*a*c)**(1/2)) / (2*a)
    return x_1, x_2
```

Notebooks

WHEN YOUR SCRIPT HAS NO
SYNTAX ERRORS AND YOU
ARE ABOUT TO RUN IT BUT..



PEP8...

Suivre les conventions PEP 8 plus ces spécificités :

Organisation du code

- Divisez votre code en cellules logiques et commentez chaque cellule pour expliquer ce qu'elle fait
- Utilisez des en-têtes markdown pour décrire le but et la fonction de chaque section du notebook.

Imports

Placez tous les imports en début de notebook

Style du code

Utilisez les capacités d'affichage de Jupyter pour montrer des exemples d'utilisation.

Nettoyage du code

- Supprimez le code inutile ou obsolète
- Évitez d'avoir des cellules vides ou non utilisées dans le notebook
- Utilisez le menu "Kernel" pour redémarrer et effacer les sorties du notebook avant de le partager



Conclusion

La lisibilité est souvent **sous-estimée** mais c'est l'une des qualités les plus **essentielles** d'un bon code.

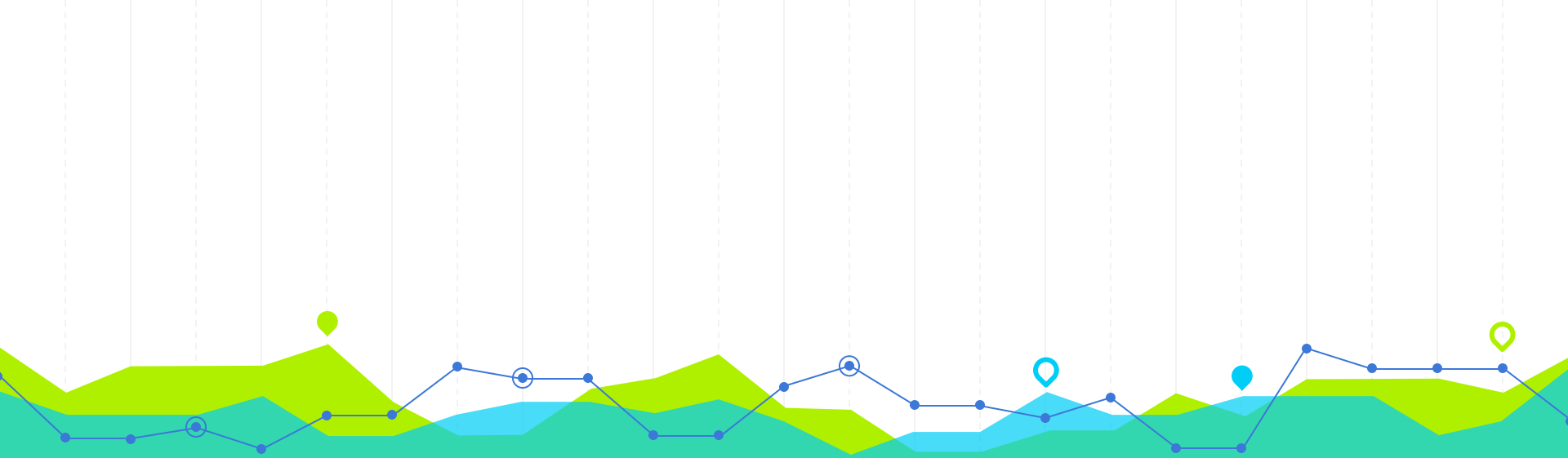
Un code lisible est plus facile à **maintenir**, **déboguer**, et **étendre**.

En insistant sur son importance **dès le début du parcours d'apprentissage**,
on instaure de **bonnes habitudes** chez les développeurs.

Pour aller plus loin :

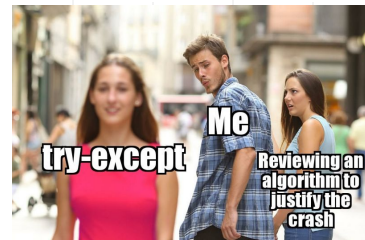
<https://peps.python.org/pep-0008/>

<https://realpython.com/documenting-python-code/>



3. Gérer les Exceptions

Erreurs VS Exceptions



Erreurs : problèmes que l'on ne peut pas gérer (ex : une erreur de syntaxe dans le code) et qui doit être corrigé dans le code

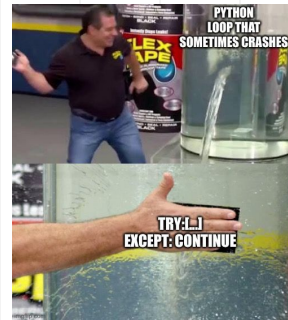
Exceptions

- événements inattendus qui se produisent pendant l'exécution du programme et peuvent être traitées à l'exécution pour éviter l'arrêt du programme
- Contrairement aux erreurs, les exceptions peuvent être "capturées" ou "gérées" à l'aide des blocs `try` et `except`. Si elles ne sont pas gérées, elles entraîneront l'arrêt du programme.

```
try:
    x = int("hello")
except ValueError:
    print("Oops! Ce n'est pas un nombre valide.")
```

"Les erreurs ne doivent jamais passer silencieusement. À moins d'être explicitement réduites au silence.", PEP8

Comment et quand utiliser try, except, else, et finally



try :

- Placez le code susceptible de lancer une exception dans le bloc `try`
- À utiliser quand une partie du code peut lancer une exception, mais vous ne voulez pas que cette erreur stoppe l'exécution du programme

except :

- Spécifiez le type d'exception à attraper et le code à exécuter si cette exception est lancée
- À utiliser après le bloc `try` pour attraper les erreurs

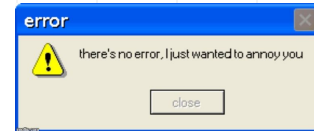
else :

- Ce bloc sera exécuté si le bloc `try` ne lève aucune exception.
- À utiliser quand vous avez du code que vous souhaitez exécuter seulement **si aucune exception** n'a été soulevée dans le bloc `try`

finally :

- Ce bloc sera exécuté **quel que soit** le résultat des blocs précédents
- À utiliser quand vous avez du code que vous devez exécuter quoi qu'il arrive, comme fermer un fichier ou une connexion à une BDD.

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Vous avez tenté de diviser par zéro.")
# Vous pouvez également attraper plusieurs
exceptions :
except (ZeroDivisionError, TypeError):
    print("Une erreur s'est produite.")
# Ou attraper toutes les exceptions (bien que cela
ne soit pas recommandé sauf si nécessaire, car
cela peut masquer des bugs) :
except Exception as e:
    print(f"Erreur rencontrée : {e}")
else:
    print("Tout s'est bien passé !")
finally:
    print("Ce bloc sera toujours exécuté.")
```



Comment créer et lever des exceptions personnalisées ?

Etape 1 : Définir votre exception personnalisée

Définissez une nouvelle classe qui **hérite** de la classe de base `Exception` ou de l'une de ses sous-classes :

```
class MonErreur(Exception):  
    pass
```

Ajouter des méthodes d'initialisation pour accepter des arguments et générer des **messages d'erreur personnalisés** :

```
class ErreurAvecMessage(Exception):  
    def __init__(self, message="Une erreur est survenue"):  
        self.message = message  
        super().__init__(self.message)
```



comment créer et lever des exceptions personnalisées

Étape 2: Lever votre exception

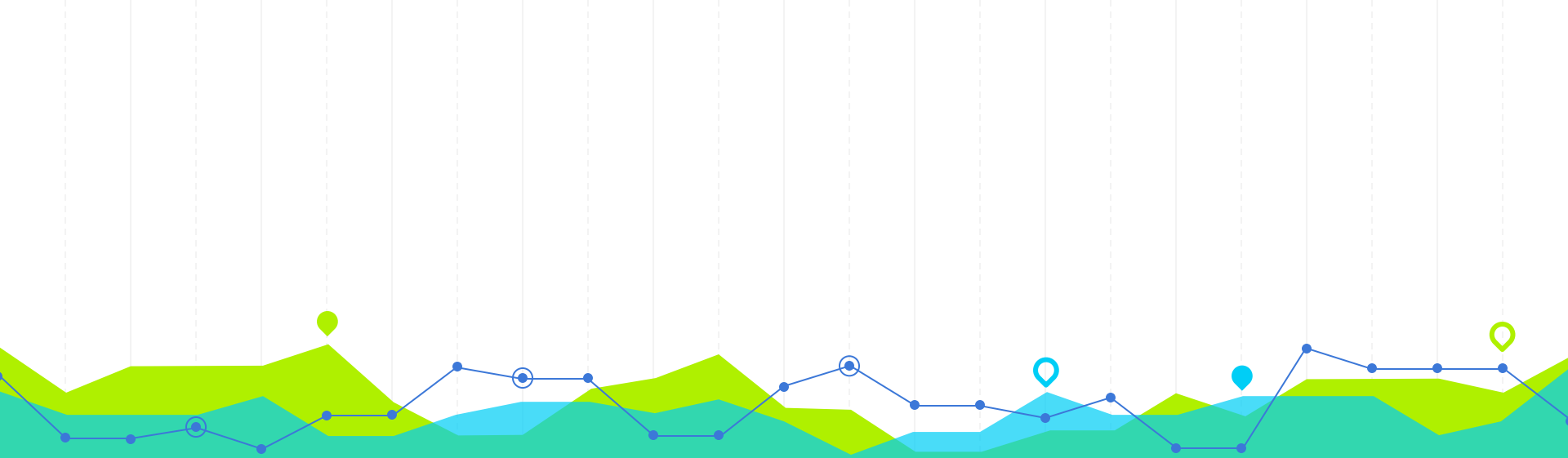
Vous pouvez "lever" votre exception (c'est-à-dire l'émettre) en utilisant le mot-clé `raise`.

```
def ma_fonction():  
    # ... votre logique ...  
    raise MonErreur( "Une erreur spécifique est survenue!" )
```

utiliser des exceptions personnalisées pour des cas spécifiques qui ne sont pas couverts par les exceptions intégrées de Python

La fonction `verifier_age` vérifie si l'âge donné est dans une plage acceptable. Si ce n'est pas le cas, elle lève une `ErreurAgeInvalide`:

```
class ErreurAgeInvalide (Exception):  
    def __init__(self, age, message= "L'âge fourni est invalide" ):  
        self.age = age  
        self.message = f"{message}.  ge donné: {age}"  
        super().__init__(self.message)  
  
    def verifier_age(age):  
        if age < 0 or age > 150:  
            raise ErreurAgeInvalide(age)  
        else:  
            print( f"L'âge {age} est valide!" )  
  
try:  
    verifier_age(200)  
except ErreurAgeInvalide as e:  
    print(e)
```



3. Logger son application

Le logging



Le logging, ou journalisation, est le processus qui consiste à enregistrer des messages (logs) pour décrire les événements qui ont lieu pendant l'exécution d'un programme.

Ces messages peuvent concerner des informations, des avertissements, des erreurs, etc.

Pourquoi est-il important ?

- **Diagnostic** : aider à diagnostiquer des problèmes imprévus dans votre application
- **Surveillance** du comportement de votre application en temps réel ou après son exécution
- **Audit** : pour certaines applications (notamment les applications bancaires ou celles qui manipulent des données sensibles), la journalisation est obligatoire pour respecter des exigences légales



Print VS Logging

Méthode "print"

- **Avantages** : simple à utiliser, bon pour un débogage rapide et un retour immédiat
- **Inconvénients** :
 - **Pas adapté** pour le code en production
 - Difficile à **filtrer** ou à désactiver sélectivement
 - Entraîne une **pollution** de la sortie standard

Méthode "logging"

- **Avantages** :
 - **Flexible** : possibilité de définir des niveaux de gravité, des formateurs, des gestionnaires, etc.
 - **Scalable** : s'adapte à des applications petites ou de grande envergure
 - **Adapté pour le code de production** : enregistrer des logs dans des **fichiers**, des bases de données, des mails, des services distants, etc. avec des **détails** tels que le moment exact ou dans quel fichier/module l'événement s'est passé
- **Inconvénients** :
 - Nécessite une initialisation et une configuration
 - Peut sembler complexe pour les débutants



Comment initialiser un logger de base

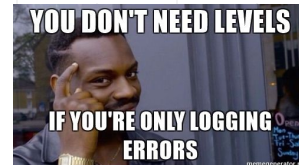
```
import logging

logging.basicConfig(level=logging.INFO) # INFO est le niveau par défaut

logging.debug("Ceci est un message de niveau DEBUG")
logging.info("Ceci est un message de niveau INFO")
logging.warning("Ceci est un message de niveau WARNING")
logging.error("Ceci est un message de niveau ERROR")
logging.critical("Ceci est un message de niveau CRITICAL")
```

vous ne verrez que les messages de niveau **INFO**, **WARNING**, **ERROR**, et **CRITICAL** car nous avons défini le niveau de base du logger sur **INFO**.

Les messages **DEBUG** sont en dessous de ce niveau et ne sont donc pas affichés.



Niveaux de gravité (Severity Levels)

différents niveaux pour indiquer la sévérité d'un événement et ajuster la granularité de l'information que vous souhaitez enregistrer

1. **DEBUG** (Valeur : 10)
 - pour les petites précisions, principalement pour le débogage
 - Exemple : `logging.debug("Variable x = %s", x)`
2. **INFO** (Valeur : 20)
 - pour confirmer que les choses fonctionnent comme prévu
 - Exemple : `logging.info("L'application a démarré avec succès.")`
3. **WARNING** (Valeur : 30)
 - il s'est passé quelque chose d'inattendu ou qu'un problème peut survenir, mais que le logiciel fonctionne encore
 - Exemple : `logging.warning("Espace disque faible.")`
4. **ERROR** (Valeur : 40)
 - décrit un problème plus grave qui a empêché le logiciel d'effectuer une fonction
 - Exemple : `logging.error("Impossible de se connecter à la base de données.")`
5. **CRITICAL** (Valeur : 50)
 - indique une erreur très grave qui pourrait empêcher le logiciel de fonctionner
 - Exemple : `logging.critical("Système hors service !")`

L'intérêt principal de définir un niveau de logging est de **filtrer les messages**.

Pour définir le niveau de logging : `logging.basicConfig(level=logging.WARNING)`



Configurer et utiliser le module logging

Le module `logging` est une partie intégrante de la bibliothèque standard Python

Concepts de base

Le module `logging` est basé sur plusieurs concepts principaux :

- **Logger** : interface principale que votre code utilisera directement. Utilisé pour **créer des messages**
- **Handler** : détermine **où** le message du journal sera routé : console, fichiers, syslog, etc.
- **Formatter** : spécifie la **mise en forme** ou la mise en page du message
- **Filter** : mécanismes plus fins pour déterminer quels messages enregistrer
- **Level** : niveau de gravité du message

Configuration avancée

Les handlers

Un handler est **responsable de la distribution des messages de log à une destination spécifique**

```
import logging

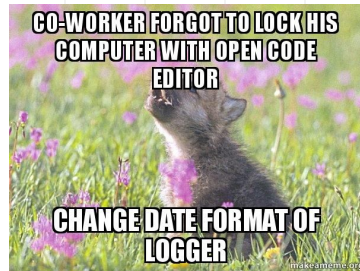
logger = logging.getLogger(__name__)

# Handler pour écrire les logs dans un fichier
file_handler = logging.FileHandler('app.log')
file_handler.setLevel(logging.ERROR) # Ne log que les erreurs et les messages plus critiques

# Handler pour afficher les logs dans la console
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.DEBUG) # Affiche tous les niveaux de logs

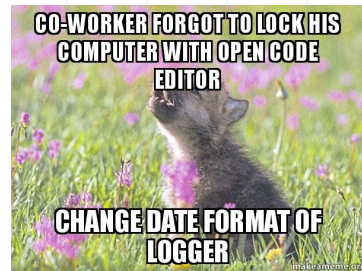
# Ajout des handlers au logger
logger.addHandler(file_handler)
logger.addHandler(console_handler)

# Test avec un log critique
logging.critical("Test critique !")
```



Configuration avancée

Les formatters



Format des messages de log

Le format ci-dessus affiche la date et l'heure, le nom du logger, le niveau du log et le message.

```
formatter = logging.Formatter( '%(asctime)s - %(name)s - %(levelname)s - %(message)s' )
```

```
# Ajout du formateur aux handlers
file_handler.setFormatter(formatter)
console_handler.setFormatter(formatter)
```

Conseil : Assurez-vous d'inclure **suffisamment d'informations** pour comprendre le contexte de chaque log, surtout si vous lisez des logs d'une **application en production**.

Les bonnes pratiques de logging



Quand logger

- **Moments clés** de l'exécution (début, fin d'une opération)
- Gestion des **erreurs** ou **exceptions**
- Événements **significatifs** (changement d'état, action de l'utilisateur)

Quand éviter de logger

- À l'intérieur de boucles exécutées fréquemment
- Des informations sensibles comme des mots de passe ou des données personnelles

```
for item in large_list:
    # Évitez cela :
    # logger.debug(f"Processing item: {item}")
    process(item)
```

Importance des messages clairs et des conventions

- **Clarté** : les messages de log doivent être **compréhensibles** même plusieurs mois après. Évitez les abréviations ou les messages trop courts.
- **Convention** : utilisez un **format standard** pour tous vos logs. Cela facilite leur lecture et leur analyse.

```
# Mauvais :
logger.info("db conn")
logger.info("qry exec")
```

```
# Bon :
logger.info("Established database connection.")
logger.info("Executed the query successfully.")
```

Les bonnes pratiques de logging



Gestion des exceptions

- **Capter des informations détaillées** : utilisez l'argument `exc_info` pour logger des exceptions avec des détails.
- **Ne pas masquer les erreurs** : assurez-vous qu'une exception loguée est **gérée** correctement (relevée, traitée, etc.).

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    logger.error("Division by zero attempted!", exc_info=True)  
    # Traitez l'exception comme nécessaire
```

Division by zero attempted!
ERROR: __main__:Division by zero attempted!



Division by zero attempted!
Traceback (most recent call last):
 File "/var/folders/kf/thbzykjl1598yzlq4chtmw6c0000gn/T/ipykernel_2618/2056549532.py", line 2, in <module>
 result = 10 / 0
ZeroDivisionError: division by zero
ERROR: __main__:Division by zero attempted!
Traceback (most recent call last):
 File "/var/folders/kf/thbzykjl1598yzlq4chtmw6c0000gn/T/ipykernel_2618/2056549532.py", line 2, in <module>
 result = 10 / 0
ZeroDivisionError: division by zero

Un bon log donne un **aperçu clair** de ce qui se passe **pendant l'exécution** et est un atout majeur lors de la résolution des problèmes.



Les pièges courants et comment les éviter

Éviter d'utiliser le root logger directement

- Il s'agit du logger de niveau supérieur fourni par défaut par le module `logging`.
- Utiliser le root logger directement peut causer des **conflits** avec d'autres bibliothèques ou morceaux de code qui l'utilisent également

Solution : Toujours créer et utiliser un logger spécifique à votre module ou application.

```
logger = logging.getLogger(__name__)
```

Prendre garde à la performance : éviter le logging dans les boucles

Écrire un log à chaque itération d'une boucle qui s'exécute des milliers de fois peut **ralentir considérablement** une application

Solution : limitez le logging dans les boucles ou envisagez d'utiliser un niveau de logging plus élevé (ex : `WARNING` plutôt que `DEBUG`)



Les pièges courants et comment les éviter

Veiller à la gestion des fichiers de log : rotation, archivage

- Les fichiers de log peuvent rapidement prendre **beaucoup d'espace disque**
- Cela peut également **rendre le débogage plus difficile**, comme chercher une aiguille dans une botte de foin.

Solution :

- Définissez une taille maximale pour chaque fichier et un nombre maximal de fichiers à conserver
- Utilisez `RotatingFileHandler` pour créer une rotation des fichiers de log
- Envisagez d'utiliser `TimedRotatingFileHandler` pour une rotation basée sur le temps

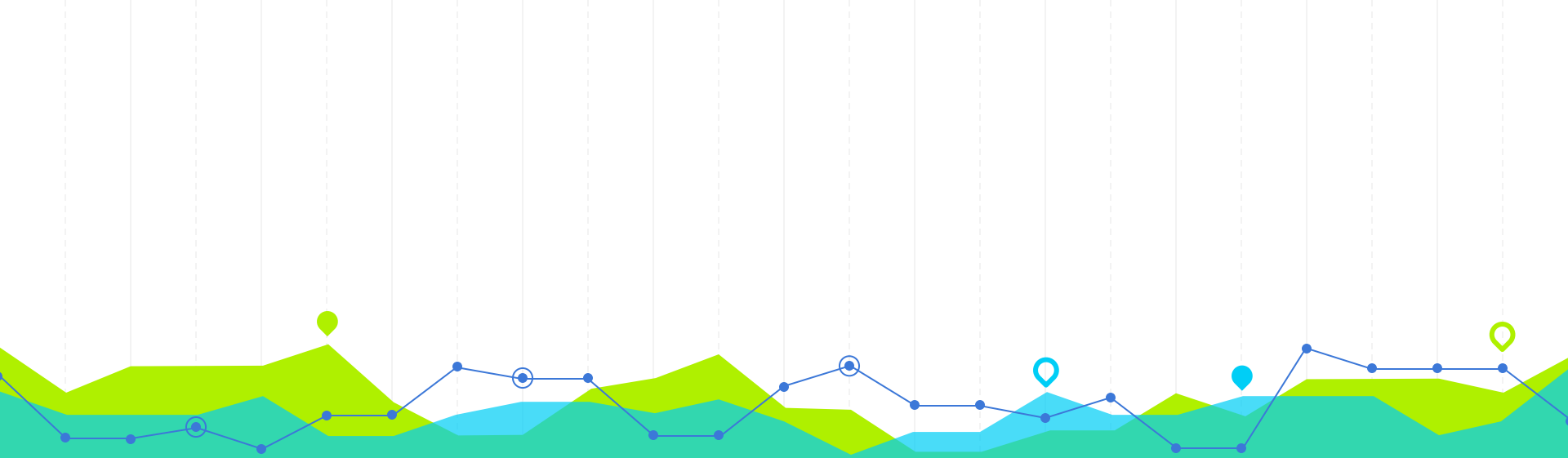
```
from logging.handlers import RotatingFileHandler
handler = RotatingFileHandler('app.log', maxBytes=5*1024*1024, backupCount=5)
logger.addHandler(handler)
```

Conclusions



Le module **logging** améliore la détection et la correction des problèmes, ainsi que la surveillance générale de l'application en production.

Un bon logging est **l'un des piliers d'un code de production propre et optimisé**.
Il facilite la maintenance, améliore la transparence des opérations et contribue à la détection précoce des problèmes.



5. sécuriser son code

Sécuriser les informations sensibles (mots de passe, clés API)



Ne jamais stocker les informations sensibles directement dans le code

Utilisez des Variables d'Environnement

Vous pouvez **définir** ces variables sur votre machine ou dans le contexte de déploiement de votre application (ex : fichier .env)

Vous pouvez accéder à ces variables en utilisant la bibliothèque `os` :

```
import os
api_key = os.environ.get('MY_API_KEY')
```

Utilisez le module `secrets` pour les opérations cryptographiques:

Python fournit le module `secrets` pour générer des tokens sécurisés pour des tâches telles que les mots de passe temporaires ou les liens de réinitialisation

```
from random import randint
token_hex = secrets.token_hex(16) # affiche '2da93b0e184e5e00f57fa0cdfde4f0e7'
```

Utilisez des Gestionnaires de Secrets

Il existe des solutions comme HashiCorp's Vault, AWS Secrets Manager, ou Azure Key Vault pour stocker, gérer et contrôler l'accès aux secrets

```
# environment variables defined in .env file
GCP_PROJECT_ID=my-project-id
SERVICE_ACCOUNT_FILE=path/to/serviceAccountCredentials
STORAGE_BUCKET_NAME=my-super-important-data
```

```
import os
from dotenv import load_dotenv

load_dotenv()
```

```
GCP_PROJECT_ID = os.getenv('GCP_PROJECT_ID')
SERVICE_ACCOUNT_FILE =
os.getenv('SERVICE_ACCOUNT_FILE')
STORAGE_BUCKET_NAME =
os.getenv('STORAGE_BUCKET_NAME')
```

Sécuriser les informations sensibles (mots de passe, clés API)



Chiffrer vos Données

Vous pouvez utiliser des bibliothèques comme `cryptography` pour faciliter le chiffrement et le déchiffrement.

Restreignez l'Accès:

- assurez-vous que seules les personnes ou les services nécessaires aient accès aux informations sensibles
- pensez à utiliser `.gitignore` pour éviter de suivre les fichiers contenant des données sensibles

Révision du Code

Pour vous assurer que les informations sensibles ne sont pas exposées accidentellement (ex : dans CI)

Mise à jour régulière

- Assurez-vous que votre code, dépendances et outils sont toujours à jour pour les dernières corrections de sécurité
- Changer régulièrement les mots de passe, les clés d'API et autres secrets pour minimiser l'impact d'une éventuelle fuite

Formation

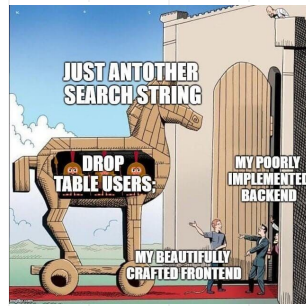
Sensibilisez les membres de votre équipe à l'importance de la sécurité et **formez-les** aux bonnes pratiques

Audits de Sécurité

Si possible, faites auditer votre code et vos infrastructures **par des experts** en sécurité pour identifier les éventuelles vulnérabilités



Éviter les vulnérabilités courantes en Python



Injection (ex : SQL Injection)

N'utilisez jamais de concaténation de chaînes pour créer des requêtes SQL. Utilisez toujours des **requêtes paramétrées**

Mauvaise pratique

```
query = "SELECT * FROM users WHERE username='" + username + "'" AND password='" + password + "'"
```

Bonne pratique avec SQLite, par exemple

```
cursor.execute("SELECT * FROM users WHERE username=? AND password=?", (username, password))
```

Exécution de code arbitraire

- Évitez d'utiliser des fonctions comme `eval()` et `exec()` qui exécutent du code à partir de chaînes de caractères
- Si vous devez absolument les utiliser, assurez-vous de bien **valider** et de **nettoyer** les entrées

Exposition de données sensibles

- Ne stockez jamais des informations sensibles en clair. Utilisez des méthodes de hachage + grain de sel.
- Évitez d'exposer des **détails d'erreur détaillés** à l'utilisateur

Éviter les vulnérabilités courantes en Python



Utilisation de bibliothèques obsolètes

Désérialisation non sécurisée:

- attention lorsque vous utilisez des fonctions comme `pickle.loads()`. Les attaquants peuvent exploiter la désérialisation pour exécuter du code arbitraire.
- Préférez des formats plus sûrs comme JSON et des bibliothèques comme `json.loads()`.

Manque de validation des entrées

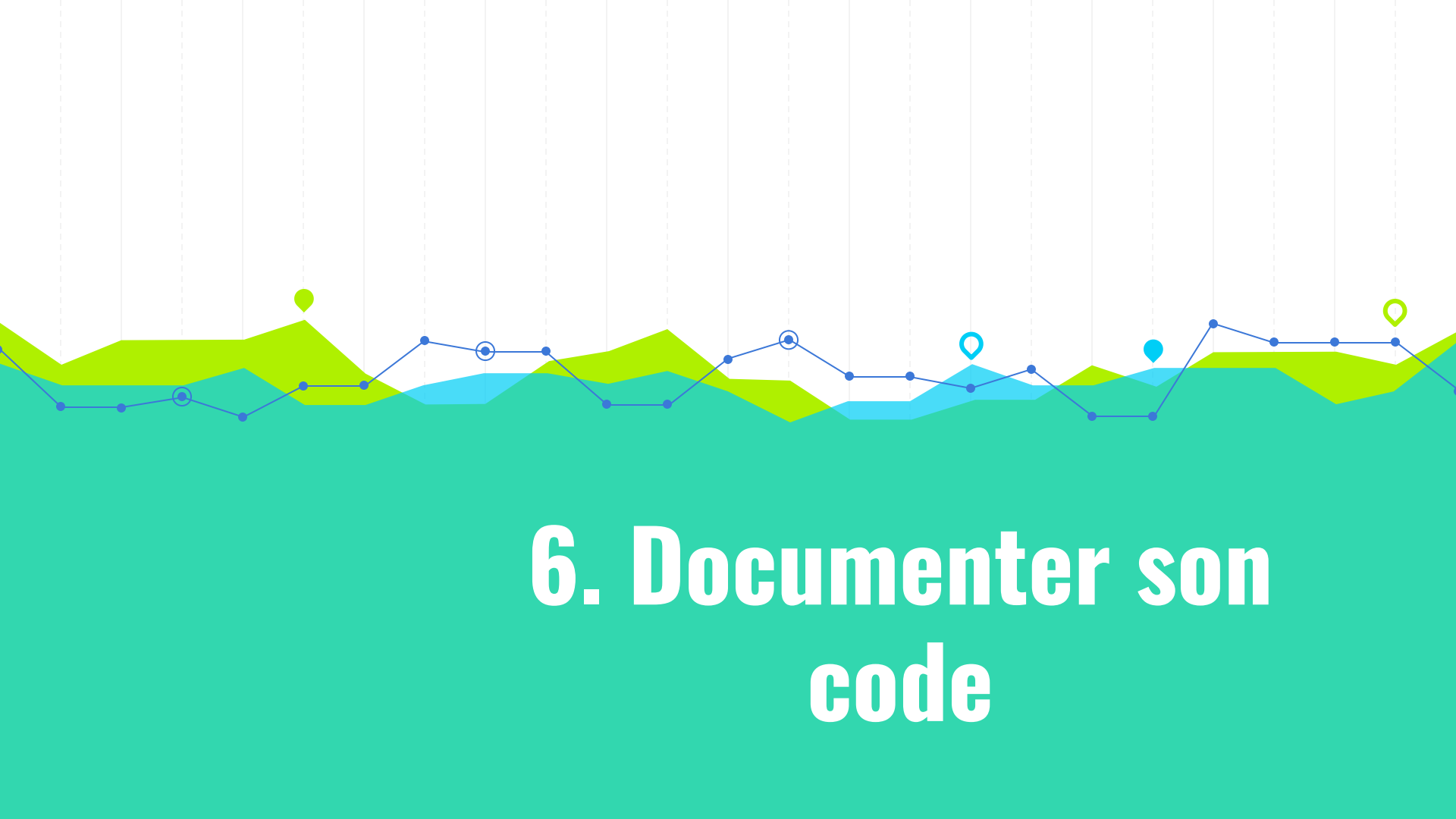
- **Validez toujours** les entrées reçues, que ce soit depuis un formulaire web, un fichier ou une autre source (via des expressions régulières ou d'autres méthodes)

Exposition des services à des attaquants

- Assurez-vous que les **ports non nécessaires sont fermés** et que les services tournant sur ces ports sont sécurisés
- Évitez d'utiliser des **configurations par défaut** pour les bases de données et autres services

Non respect des meilleures pratiques HTTPS

- Si votre application est une application web, assurez-vous d'utiliser HTTPS pour protéger les données en transit
- Utilisez des mécanismes comme HSTS (HTTP Strict Transport Security) pour forcer l'utilisation de HTTPS



6. Documenter son code

Pourquoi documenter ?



L'importance de la documentation

- **Lisibilité** : Même si un code est bien écrit, sans documentation, il peut être difficile à comprendre pour un autre développeur ou pour vous-même à l'avenir.
- **Maintenance** : Facilite la mise à jour et la détection d'erreurs. Un code bien documenté permet d'identifier rapidement les zones à modifier et les implications de ces modifications.
- **Collaboration** : Dans un environnement d'équipe, la documentation assure que tous les membres comprennent le fonctionnement et l'intention du code, permettant une collaboration efficace.

Points clés à retenir :

1. La documentation est une partie intégrante du processus de développement et ne devrait pas être une réflexion après coup.
2. Un code bien documenté est souvent synonyme de qualité et de professionnalisme.
3. La documentation aide non seulement les autres, mais aussi vous-même à l'avenir.



Types hinting (annotations de type)

Permet d'indiquer **explicitement** le **type attendu** d'une variable, d'un argument de fonction ou du retour d'une fonction.
Ne modifie pas le comportement du programme (Python reste un langage à **typage dynamique**), mais il offre une meilleure **lisibilité** et peut être utilisé avec des **outils externes** pour vérifier la cohérence des types (**mypy** ou **traitlets**)

```
def greet(name: str) -> str:  
    return "Hello, " + name
```

Bonnes pratique

- N'abusez pas des annotations de type : utiles pour la clarté, mais inutilement verbales dans certains contextes.
- Les annotations de type sont pour les développeurs et les outils, pas pour l'interpréteur Python
- **mypy** peut ralentir le cycle de développement s'il est utilisé à chaque fois. Utilisez-le judicieusement.



Docstrings

Utilité

- Assure de **comprendre** rapidement l'objectif de la fonction
- Fournit une description claire des paramètres attendus, du type de retour, et des exceptions éventuellement levées
- Facilite la **génération automatique de la documentation** via des outils

Outils pour exploiter les docstrings

- **pydoc** génère automatiquement la documentation à partir des docstrings.
 - exécutez simplement `pydoc nom_du_module` dans le terminal.
- **Sphinx**: un outil puissant pour générer une documentation complète

Toujours inclure des **docstrings**, même pour les fonctions qui semblent simples.

```
def add(a: int, b: int) -> int:
    """Additionne deux nombres.

    Args:
        a (int): Le premier nombre.
        b (int): Le second nombre.

    Returns:
        int: La somme des deux nombres.
    """
    return a + b
```

```
def greet() -> None:
    """Affiche un message de bienvenue."""
    print("Bonjour!")
```

```
class SimpleClass:
    """Class docstrings go here."""

    def say_hello(self, name: str):
        """Class method docstrings go here."""
        print(f'Hello {name}')
```



Docstrings - bonnes pratiques

1. Il existe **plusieurs conventions** pour la rédaction de docstrings, telles que la convention Google, la convention NumPy et la convention reStructuredText (reST)
2. **Décrivez les paramètres et les valeurs de retour** (avec le type et une description)
3. La première ligne du docstring doit être une phrase concise décrivant la finalité de la fonction ou de la classe
4. Fournir des **exemples d'utilisation**
5. Si votre fonction ou méthode peut **lever des exceptions**, **décrivez** les situations dans lesquelles elles sont levées
6. Utiliser les **sections** "Args", "Returns", "Raises", etc. pour organiser votre docstring
7. Accéder aux docstrings à l'aide de la fonction **help()** en mode interactif.
8. Lorsque vous mettez à jour votre code, n'oubliez pas de **mettre à jour vos docstrings** pour refléter les changements
9. des outils comme **pydocstyle** peuvent vous aider à **vérifier que vos docstrings sont conformes** à certaines conventions



Module

Faciliter la compréhension de ce que fait le module dans son ensemble, ainsi que l'interaction entre ses composants.

Docstrings de module:

- Placées au début du fichier du module.
- Fournissent un résumé des principales fonctionnalités et classes du module.

```
"""This is the example module.
```

```
This module does stuff.  
"""
```

```
import os  
import sys
```



Package

Offre une vue d'ensemble du package, détaillant ses modules, ses dépendances principales et son objectif général

Quand elle est nécessaire : lorsque plusieurs modules sont regroupés pour former une bibliothèque.

Docstrings de package :

- Placées dans le fichier `__init__.py` du package.
- Mettre en avant la structure des sous-modules et la manière dont ils interagissent
- Précisez toute dépendance externe ou version spécifique de Python nécessaire pour que le package fonctionne correctement. S'il y a des parties du package qui nécessitent des dépendances optionnelles, cela doit également être documenté.



Sphinx

Sphinx est l'un des outils de documentation les plus populaires et largement utilisés dans la communauté Python, notamment en raison de sa capacité à **générer une documentation à partir des docstrings** et de son **intégration étroite avec Read the Docs**.

Installez Sphinx :

```
pip install sphinx
```

Générer la documentation:

Depuis le répertoire où se trouve `conf.py`, exécutez :

```
make html
```

Cela générera des fichiers HTML dans un dossier `_build`.
Ouvrez `_build/html/index.html` pour voir votre documentation

Sphinx

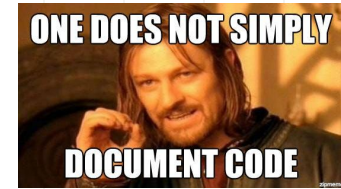


Alternatives à Sphinx:

- **MkDocs** : outil de génération de documentation rapide et simple écrit en Python. Il utilise Markdown au lieu de reStructuredText. Avec le thème Material for MkDocs, il produit une documentation très moderne et responsive
- **Doxygen** : principalement utilisé pour les projets en C++, Doxygen prend également en charge la documentation de projets en Python, C, Java, entre autres. Il génère une documentation à partir des commentaires source
- **Jekyll** : principalement utilisé comme générateur de sites statiques, Jekyll (souvent hébergé sur GitHub Pages) peut être utilisé pour la documentation, en particulier avec des plugins ou des thèmes dédiés à la documentation
- **Docusaurus** : outil de Facebook pour construire des sites de documentation. Il est basé sur React et permet une personnalisation plus avancée
- **VuePress** : génère un site statique à partir de fichiers Markdown. Il est basé sur Vue.js, ce qui signifie que vous pouvez facilement intégrer des composants Vue dans votre documentation
- **GitBook** : service en ligne offrant une interface très soignée pour écrire et publier de la documentation. Il prend en charge la collaboration en équipe et l'écriture en Markdown

Sphinx

- Ces librairies permettent d'automatiser la création d'une documentation à partir de votre codebase de manière fiable et déterministe
- Apparition dans l'écosystème de multiples librairies qui sont pensées pour fonctionner pour ou avec les LLMs
 - **Mintlify**: combine extraction automatique de code avec une génération par LLM d'exemples et de code d'utilisation de vos librairies
 - **Swimm**: approche mixte entre règles écrites à la main et LLM
 - ...
- Cela permet de maintenir votre codebase synchronisée avec les exemples d'utilisation



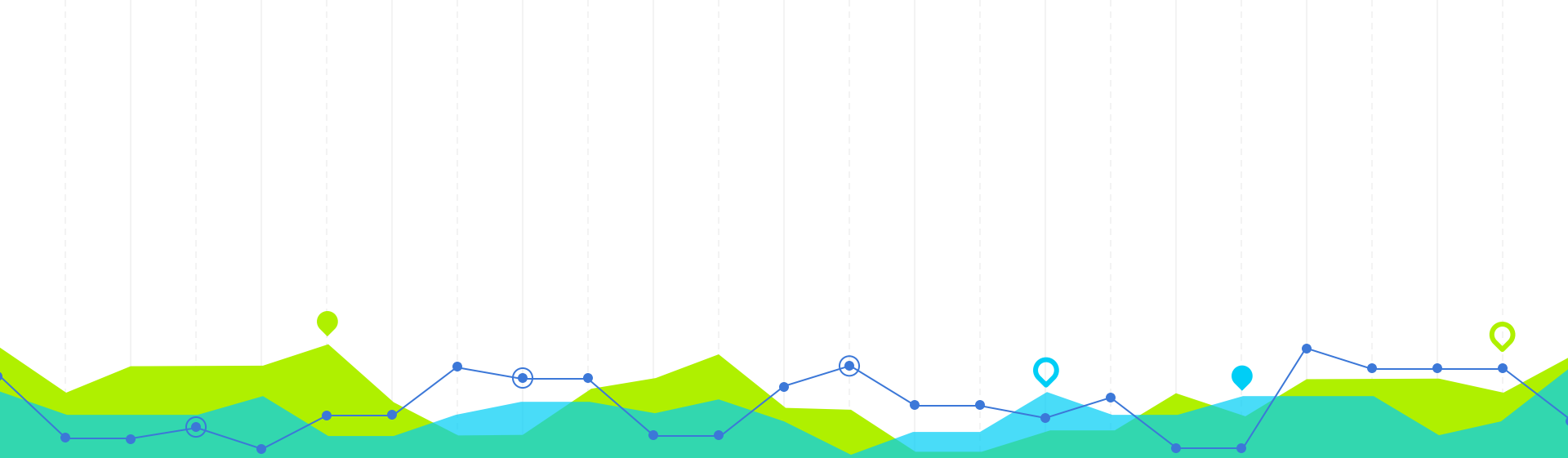
Conclusion

Commenter / documenter

- Facilite la **maintenance** et permet de **comprendre rapidement** le code
- Durée de vie plus longue du code car il moins susceptible d'être **complètement réécrit** parce que personne ne le comprend

Attention cependant

- **Trop de documentation** peut la rendre fastidieuse à lire et à maintenir
- À se concentrer sur le "**pourquoi**" plutôt que sur le "comment", car le code lui-même décrit le "comment".
- Documenter **en cours d'écriture** garantit que la documentation est à jour et cohérente avec le code
- Il est plus difficile (et souvent négligé) de revenir et de **documenter après coup**



7. Tester son code

Pourquoi tester ?



Les tests :

- garantissent que votre code **fonctionne comme prévu**
- permettent de détecter les erreurs si vous avez accidentellement introduit des erreurs suite à des modifications dans le code (**régressions**)
- peuvent servir de **documentation** complémentaire.
- augmente la **confiance**

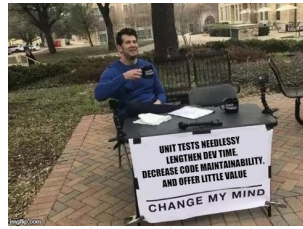
Développement dirigé par les tests (TDD) : certains développeurs utilisent les tests comme une **méthodologie** pour concevoir leur code. **Écrire d'abord les tests** permet de se concentrer sur ce que le code doit accomplir avant de se préoccuper de la manière dont il l'accomplira.

Unittest

`unittest` est le framework de test intégré à la bibliothèque standard de Python.

```
def additionner(a, b):  
    return a + b
```

```
import unittest  
from mon_module import additionner  
  
class TestAdditionnerFonction(unittest.TestCase):  
  
    def test_additionner_positif(self):  
        self.assertEqual(additionner(1, 2), 3)  
  
    def test_additionner_negatif(self):  
        self.assertEqual(additionner(-1, -2), -3)  
  
    def test_additionner_mixte(self):  
        self.assertEqual(additionner(1, -2), -1)  
  
if __name__ == '__main__':  
    unittest.main()
```



Pytest

pytest est une bibliothèque externe très populaire pour les tests en Python. Elle offre une syntaxe plus concise et une meilleure détection des tests.

```
from mon_module import additionner

def test_additionner_positif():
    assert additionner(1, 2) == 3

def test_additionner_negatif():
    assert additionner(-1, -2) == -3

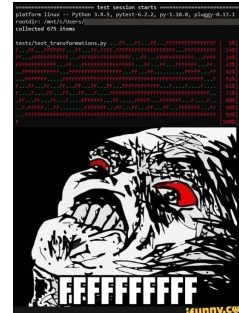
def test_additionner_mixte():
    assert additionner(1, -2) == -1
```

Avec **pytest**, vous n'avez pas besoin de classes ou de méthodes spéciales.

Vous écrivez simplement des fonctions qui commencent par **"test_"**.

Pour exécuter les tests avec **pytest**, installez **pytest** (pip install pytest) et exécutez la commande **pytest** dans votre terminal.

Choisir entre **unittest** et **pytest** dépend généralement de vos préférences personnelles et des besoins de votre projet.



Pourquoi tester ?

Supposons que vous ayez une application de blog avec une classe `Article`.
Pour simplifier, cet article aura juste un titre et un contenu :

```
# blog.py
class Article:
    def __init__(self, title, content):
        self.title = title
        self.content = content
```

Vous voulez vous assurer que les articles sont correctement initialisés :

```
# test_blog.py
import pytest
from blog import Article

# Fixture pour un article de base
@pytest.fixture
def article_default():
    return Article("Un titre", "Du contenu")

def test_article_initialisation(article_default):
    assert article_default.title == "Un titre"
    assert article_default.content == "Du contenu"

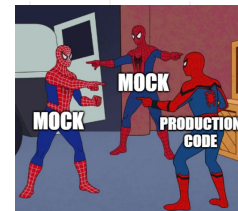
# Imaginons que nous ayons une méthode pour raccourcir le contenu
def test_article_shorten_content(article_default):
    article_default.shorten_content(5)
    assert article_default.content == "Du co..."
```



Les fixtures

Dans cet exemple, `article_default` est une **fixture**.
Chaque fois que cette fixture est passée comme **argument** à une fonction de test, la fonction de la fixture est **exécutée** et son résultat est passé à la fonction de test. Cela permet de garantir que chaque test reçoit une nouvelle instance propre de l'article, ce qui est crucial pour l'**isolation des tests**.

Les fixtures dans `pytest` peuvent également être utilisées pour des choses plus complexes, comme l'initialisation de bases de données, la création de clients de test pour des applications web, la configuration d'états spécifiques avant l'exécution d'un test, etc.



Mocking

"Mocking" est une technique utilisée en programmation pour **simuler le comportement d'objets réels** lors des tests unitaires.

C'est très utile lorsque vous voulez **isoler** la partie du code que vous testez et **ne pas dépendre d'éléments extérieurs**, tels que des bases de données, des fichiers, des appels réseau, etc.

Imaginons que vous ayez une fonction qui récupère les données d'un utilisateur depuis une base de données :

```
def get_user_from_db(user_id):  
    # Suppose que cette fonction interagit avec  
    # une base de données pour obtenir un utilisateur  
    # ...  
    return user_data
```

Si vous voulez tester cette fonction **sans interagir réellement** avec une base de données (par exemple, pour éviter de consommer des ressources ou de devoir configurer une base de données pour vos tests), vous pouvez utiliser un mock:

```
from unittest.mock import Mock  
  
# Création d'un mock  
mocked_function = Mock(return_value={"id": 1, "name": "John  
Doe"})  
  
# Maintenant, à chaque fois que vous appelez  
# 'mocked_function', elle retournera le dictionnaire spécifié  
user = mocked_function(1)  
  
print(user)  # {"id": 1, "name": "John Doe"}
```

Patching

Patcher vous permet de **remplacer temporairement** une méthode ou un attribut d'un module/objet par un **mock** pendant vos tests.

Exemple de Patching

Supposons que vous avez le code suivant dans un fichier `user_service.py`:

```
import requests

def get_user_from_api(user_id):
    response = requests.get(f"https://api.example.com/users/{user_id}")
    return response.json()
```

Si vous voulez tester `get_user_from_api` sans faire de véritables appels à l'API, vous pouvez utiliser `patch` pour remplacer `requests.get` par une version mockée.

```
from user_service import get_user_from_api
from unittest.mock import patch

def test_get_user_from_api():
    # patch pour remplacer `requests.get` par un mock
    with patch('user_service.requests.get') as mocked_get:
        mocked_get.return_value.json.return_value = {
            "id": 1,
            "name": "John Doe"
        }
        user = get_user_from_api(1)
        assert user == {"id": 1, "name": "John Doe"}

test_get_user_from_api()
```

Au lieu d'appeler le vrai `requests.get`, notre test utilise le mock que nous avons fourni. Cela permet de s'assurer que nos tests sont **rapides**, **fiables** et **indépendants** des dépendances externes.

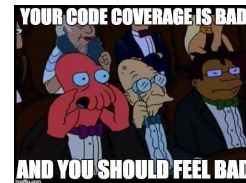
I don't often test my code.

But when I do,
I do it in
Production.

someecards
USER CARD



Couverture de code avec coverage.py



Métrique pour mesurer la **proportion** de code d'un programme **qui est exécutée** lors de l'exécution des tests.

Utile pour identifier les **parties du code** qui pourraient nécessiter des **tests supplémentaires** et pour **évaluer** la qualité des tests existants.

Coverage.py

1. **Instrumentation** : lors de l'exécution des tests, **coverage.py** **surveille** votre code pour savoir quelles parties ont été exécutées.
2. **Rapport** : Après avoir exécuté les tests, vous pouvez demander à **coverage.py** de produire un rapport montrant quelles parties du code ont été exécutées et lesquelles ne l'ont pas été.

Installation : `pip install coverage`

Exécution des tests avec coverage

```
coverage run test_program.py
```

Générer un rapport :

```
coverage report -m
```

Ceci affiche un rapport dans la console, indiquant pour chaque fichier, le pourcentage de couverture, le nombre de lignes couvertes et non couvertes, etc.

Rapport HTML

```
coverage html
```

Ouvrez htmlcov/index.html dans un navigateur.

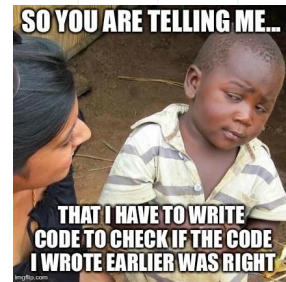
```
===== test session starts =====
platform linux -- Python 3.8.5, pytest-6.2.1, py-1.10.0, pluggy-0.13.1 -- /home/chito/Documents/Projects/happy-fin-api/app/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/chito/Documents/Projects/happy-fin-api/app
collected 10 items

api/tests.py::test_version PASSED
api/tests.py::test_signin_success PASSED
api/tests.py::test_signin_failed_bad_username PASSED
api/tests.py::test_signin_failed_bad_password PASSED
api/tests.py::test_signin_failed_no_password PASSED
api/tests.py::test_get_user_profile_invalid_token PASSED
api/tests.py::test_get_user_profile_no_token PASSED
api/tests.py::test_get_user_profile_success PASSED
api/tests.py::test_expired_token PASSED
api/tests.py::test_invalid_uid PASSED

===== 10 passed in 0.98s =====

Name                               Stmts  Miss  Cover
-----
api/_init_.py                       0      0  100%
api/app_configs.py                  16      0  100%
api/constants.py                     2      0  100%
api/exceptions.py                   17      2   88%
api/main.py                          30      0  100%
api/models.py                       89      6   93%
api/tests.py                        57      0  100%
TOTAL                             211      8   96%
```

TDD



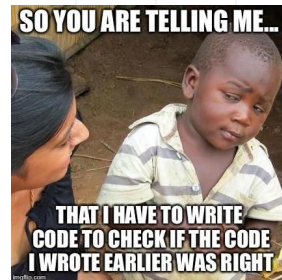
Le Développement Dirigé par les Tests (TDD, pour "Test-Driven Development") est une **approche de développement** où **l'écriture des tests précède celle du code de production**.

L'idée est de formaliser les exigences et les spécifications sous forme de tests unitaires avant d'écrire le code qui permet de satisfaire ces tests.

Le cycle TDD typique se déroule en trois étapes:

1. **Écrire un test** : avant de coder la fonctionnalité, commencez par écrire un test unitaire qui décrit ce que vous attendez de cette fonctionnalité. Ce test **échouera** car la fonctionnalité n'existe pas encore.
2. **Écrire le code minimum pour passer le test** : écrivez le code nécessaire pour que le **test réussisse**. Ne vous souciez pas de la perfection ou de l'optimisation à ce stade; l'objectif est simplement de faire passer le test.
3. **Refactoriser** : une fois le test réussi, examinez votre code. Peut-il être amélioré ou optimisé tout en conservant le passage du test ?

TDD



1. Écrire un test

```
import unittest

class TestAddition(unittest.TestCase):

    def test_addition(self):
        self.assertEqual(add(1, 2), 3)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(-1, -1), -2)
```

2. Écrire le code minimum pour passer le test

```
def add(a, b):
    return a + b
```

3. Refactoriser

Dans des scénarios plus complexes, vous pourriez trouver des moyens d'optimiser le code ou de le rendre plus lisible.

Après chaque modification, réexécutez les tests pour vous assurer que tout fonctionne toujours comme prévu.

L'avantage du TDD est qu'il vous permet de vous concentrer sur ce que vous voulez que votre code accomplisse (les **spécifications**) avant de vous préoccuper de la **manière dont il le fait**. De plus, vous construisez une **suite de tests robuste** qui peut être exécutée à chaque modification, **assurant ainsi que les fonctionnalités précédemment développées fonctionnent toujours comme prévu**.

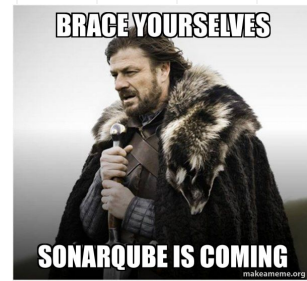


Logiciel libre de **qualimétrie en continu** de code.

Il aide à la **détection**, la **classification** et la **résolution** de défaut dans le code source, permet d'identifier les **duplications** de code, de mesurer le **niveau de documentation** et connaître la **couverture de test** déployée.

SonarQube permet une **surveillance continue** de la qualité du code grâce à son interface web permettant de voir les défauts de l'ensemble du code et ceux ajoutés par la nouvelle version.

Le logiciel peut être interfacé avec un système d'automatisation comme **Jenkins** au autre CI pour inclure l'analyse comme une extension du développement.



Python /

master

Quality Gate **Passed**

New Code Overall Code

New Code: since about 1 month ago

Reliability

0 Bugs

A

Maintainability

574 Code Smells

B

Security

0 Vulnerabilities

A

Security Review

0 Security Hotspots

A

Coverage

76% Coverage

On 21k New Lines to cover

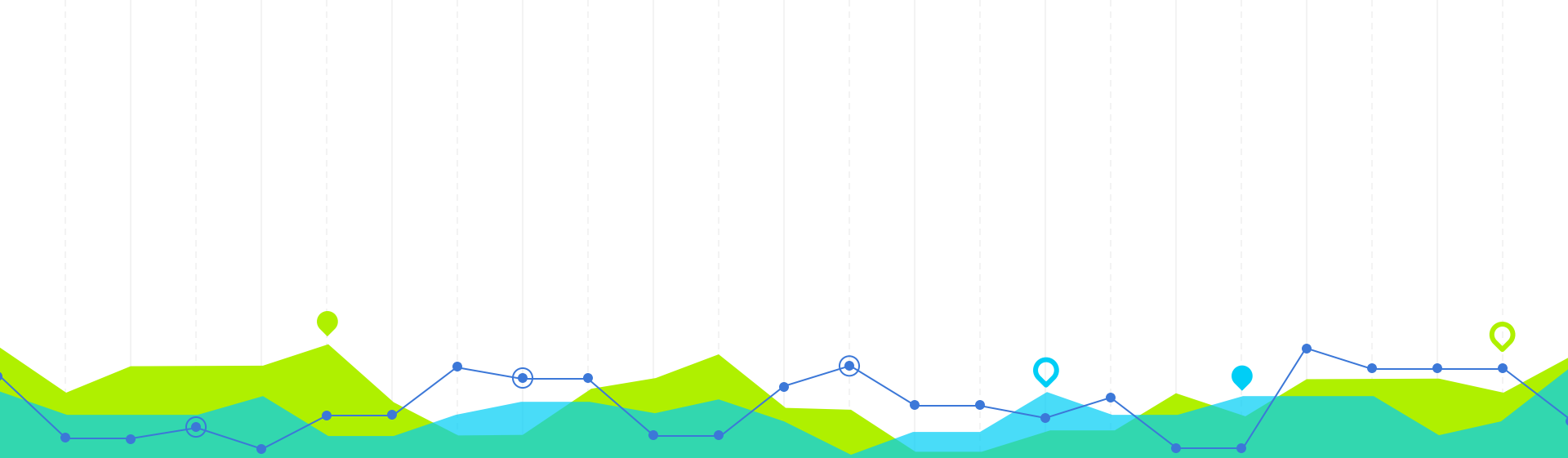


Duplications

0.0% Duplications

On 46k New Lines





Assistants de code

Fichiers de contexte

- Pour qu'un assistant de code puisse vous aider à ajouter des fonctionnalités dans votre code, vous devez lui fournir un contexte pertinent et adapté à la bonne réalisation de la tâche
- Dans les projets modernes, on retrouve souvent plusieurs fichiers avec des noms particuliers:
 - AGENTS.md
 - CLAUDE.md

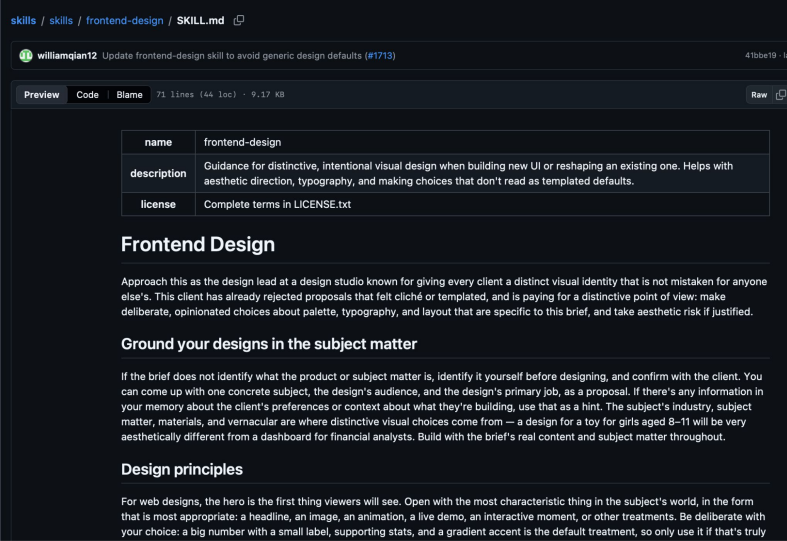


Ce qu'il faut y mettre : commandes de build/test/lint exactes, conventions non évidentes (ex. "on utilise Pixi et pas pip"), contraintes d'architecture, zones interdites (*/legacy/* à ne jamais toucher), conventions de commit propre à votre entreprise ou projet

Ce qu'il ne faut PAS y mettre : les règles de style que Black/Ruff imposent déjà, les banalités type "écris du code propre" -> ça gaspille le budget d'attention du modèle sans effet mesurable

Skills réutilisables

- Distinct du fichier de contexte global : des modules de type SKILL.md invocables à la demande (manuellement ou automatiquement via leur description)
- Encapsulent une pratique métier précise: un style de génération de rapports, une checklist de revue de sécurité, des tests



The screenshot shows a web application with a dark theme. At the top, there's a breadcrumb trail: 'skills / skills / frontend-design / SKILL.md'. Below this, a user profile 'williamqian12' is shown next to a title 'Update frontend-design skill to avoid generic design defaults (#1713)'. A status '41bbce19 - 1h' is on the right. A tab bar contains 'Preview', 'Code', and 'Blame', with 'Preview' selected. The main content area displays a table with skill metadata:

name	frontend-design
description	Guidance for distinctive, intentional visual design when building new UI or reshaping an existing one. Helps with aesthetic direction, typography, and making choices that don't read as templated defaults.
license	Complete terms in LICENSE.txt

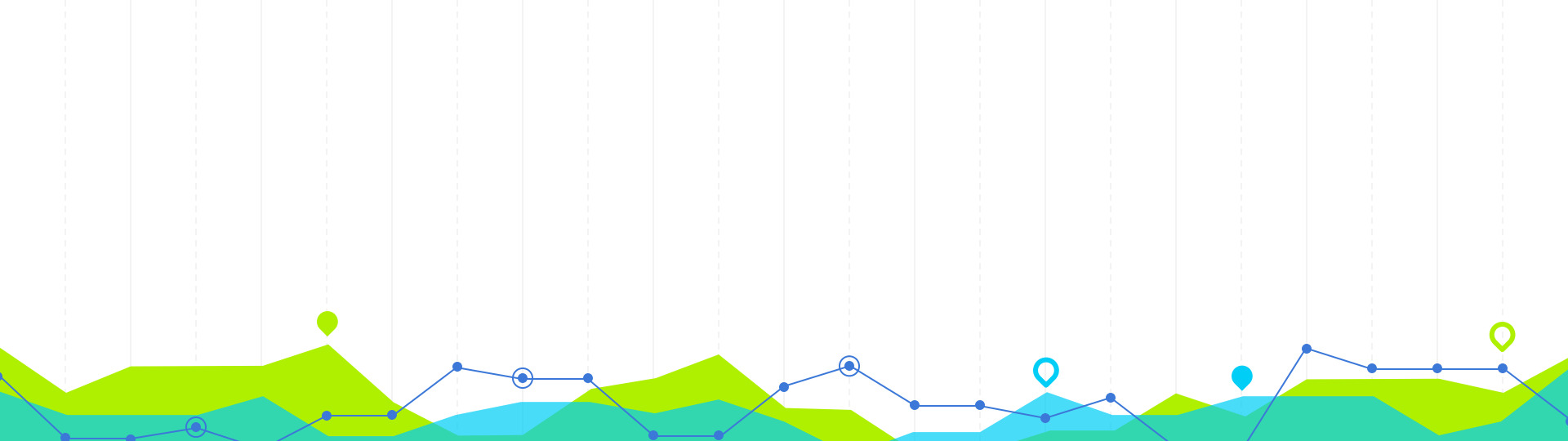
Below the table, the section 'Frontend Design' is followed by a paragraph: 'Approach this as the design lead at a design studio known for giving every client a distinct visual identity that is not mistaken for anyone else's. This client has already rejected proposals that felt cliché or templated, and is paying for a distinctive point of view: make deliberate, opinionated choices about palette, typography, and layout that are specific to this brief, and take aesthetic risk if justified.'

The next section is 'Ground your designs in the subject matter', followed by a paragraph: 'If the brief does not identify what the product or subject matter is, identify it yourself before designing, and confirm with the client. You can come up with one concrete subject, the design's audience, and the design's primary job, as a proposal. If there's any information in your memory about the client's preferences or context about what they're building, use that as a hint. The subject's industry, subject matter, materials, and vernacular are where distinctive visual choices come from — a design for a toy for girls aged 8–11 will be very aesthetically different from a dashboard for financial analysts. Build with the brief's real content and subject matter throughout.'

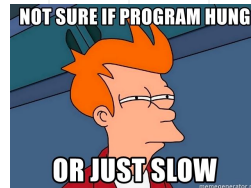
The final section is 'Design principles', followed by a paragraph: 'For web designs, the hero is the first thing viewers will see. Open with the most characteristic thing in the subject's world, in the form that is most appropriate: a headline, an image, an animation, a live demo, an interactive moment, or other treatments. Be deliberate with your choice: a big number with a small label, supporting stats, and a gradient accent is the default treatment, so only use it if that's truly

Outils déterministes vs probabilistes

- Les éléments que l'on vient de voir permettent de guider les assistants de code vers ce que vous souhaitez
- Cependant, pour des actions déterministes comme le formatting, pas besoin de faire appel à un LLM
 - pas d'ambiguïté dans l'output -> même entrée, même sortie
 - pas d'appel à un LLM donc coûts réduits au global
- Le typehinting permet également d'ajouter un contexte supplémentaire aux inputs/ouputs de vos fonctions et aide un assistant de code à générer un code cohérent avec le reste de votre projet



Introduction à l'optimisation



Introduction à l'optimisation

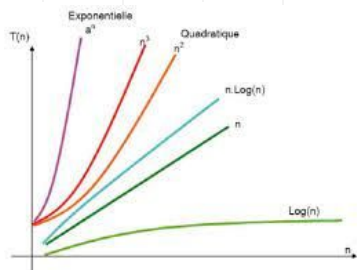
L'optimisation du code consiste à améliorer son efficacité, soit en **accélérant son exécution**, soit en **réduisant l'utilisation des ressources**.

- Impact direct sur l'expérience utilisateur (temps de réponse, utilisation des ressources).
- Implications financières : économies sur les coûts d'infrastructure, en particulier pour les applications à grande échelle.

Optimisation pour la performance : rendre le code plus rapide ou moins gourmand en ressources. Cela peut parfois rendre le code moins lisible ou plus complexe.

Exemple : Utiliser une compréhension de liste au lieu d'une boucle for traditionnelle pour réduire le temps d'exécution.

Donald Knuth : "L'optimisation prématurée est la racine de tous les maux".
Il ne faut pas optimiser sans avoir identifié au préalable où se situent les réels goulots d'étranglement.



Outils pour optimiser

- **Profiling** pour détecter les goulots d'étranglement :
 - avec `cProfile` et `profile` pour avoir le temps d'exécution et nombre d'appels de chaque fonction
 - `snakeviz` (un navigateur pour visualiser les résultats du profiling)
 - `Py-Spy` (un profiler d'échantillonnage en temps réel pour Python)
- Utilisation de **threads** :
 - avec `threading` et `queue`
 - **Multi-threading** avec `multiprocessing` et `concurrent.futures.ProcessPoolExecutor`
 - **Programmation asynchrone** pour exécuter plusieurs tâches en utilisant un seul thread en alternant entre elles : `asyncio`, `aiohttp`, `aiomysql`
- **Paralléliser** :
 - des opérations avec `Joblib` ou `Dask`
 - plus efficace en contournant le GIL avec `Cython` et `Numba`
- **Intégration de code** en C/C++
- Utilisation du **cache** :
 - avec `lru_cache` de `functools`
 - avec la mémoïsation / programmation dynamique
- Ne pas enchaîner les loop en python
- Travailler en **tensor** (via numpy, Tensorflow ou PyTorch) plutôt qu'en structure complexes comme des DataFrames Pandas

